챕터 6. Kubernetes 운영하기

며칠 뒤, 오픈이는 진행 중인 프로젝트를 배포하기 전 마지막으로 한 번 더 코드 리뷰를 하고 있었습니다. 그러던 중 Dockerfile의 환경 변수 코드가 눈에 띄었습니다.

```
ENV MYSQL_PASSWORD=metacoding1234
```

데이터베이스 비밀번호가 코드에 그대로 노출되어 있었습니다.

'잠깐. 이대로 빌드하면 비밀번호가 이미지에 그대로 남는데.'



그림 6-1. Dockerfile에 적힌 비밀번호가 이미지에 그대로 담겨 배포된다

오픈이가 옆자리 선배에게 묻자, 선배가 의자를 돌려 오픈이를 봤습니다.

선배 : "테스트 환경에서는 괜찮지만, 실제 운영 환경에 배포할 때는 두 가지를 주의해야 해요. 먼저 비밀번호 같은 설정값은 자주 바뀔 수 있으니, Dockerfile에 직접 적지 말고 **이미지 밖으로 분리**해서 따로 관리해야 해요.
그리고 Pod는 언제든 종료될 수 있어서, Pod 안에 데이터를 저장하면 안 돼요. 보관해야 할 데이터는 **Pod 외부의 저장소**에 따로 저장해야 해요. 설정과 데이터 모두 컨테이너 밖으로 안전하게 빼 두는 게 핵심이죠."

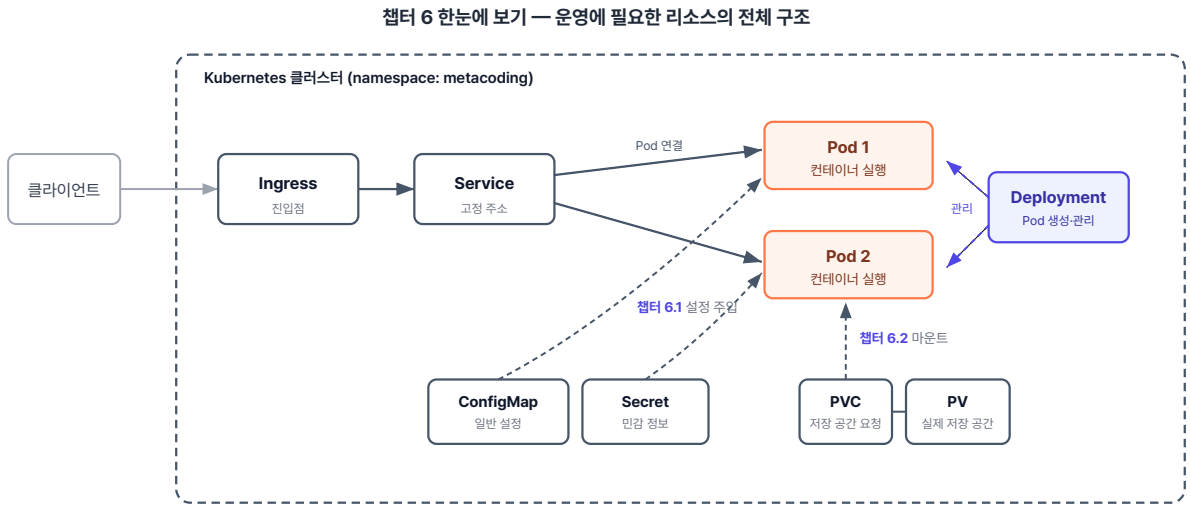


그림 6-2. 챕터 6 한눈에 보기 - 운영에 필요한 리소스의 전체 구조

실습 준비. 예제 코드

이 책의 실습 코드는 GitHub 레포 하나에 챕터별 폴더로 담겨 있습니다. 아래 명령으로 레포를 한 번 git clone 합니다. 이후 챕터마다 해당 폴더로 이동해 실습합니다.

터미널 **예제 코드 클론**

```
git clone https://github.com/metacoding-10-linux-docker/start
```

이번 챕터는 `ex13` ~ `ex15` 폴더를 사용합니다.

완성된 코드는 아래 주소에서 확인하세요.

터미널 **완성 코드**

```
https://github.com/metacoding-10-linux-docker/final
```

6.1 ConfigMap·Secret - 설정과 비밀번호를 이미지 외부로

컨테이너 이미지에 데이터베이스 주소나 비밀번호를 직접 적어 두면, 설정값이 바뀔 때마다 이미지를 새로 빌드해야 합니다. 운영 환경에서 설정 하나를 바꾸기 위해 매번 이미지를 다시 빌드하고 배포하는 것은 비효율적입니다.

그래서 쿠버네티스에는 컨테이너 이미지와 환경 변수를 분리하기 위한 **ConfigMap**과 **Secret**이라는 리소스가 있습니다. ConfigMap에는 일반 설정값을, Secret에는 민감한 정보를 저장합니다.

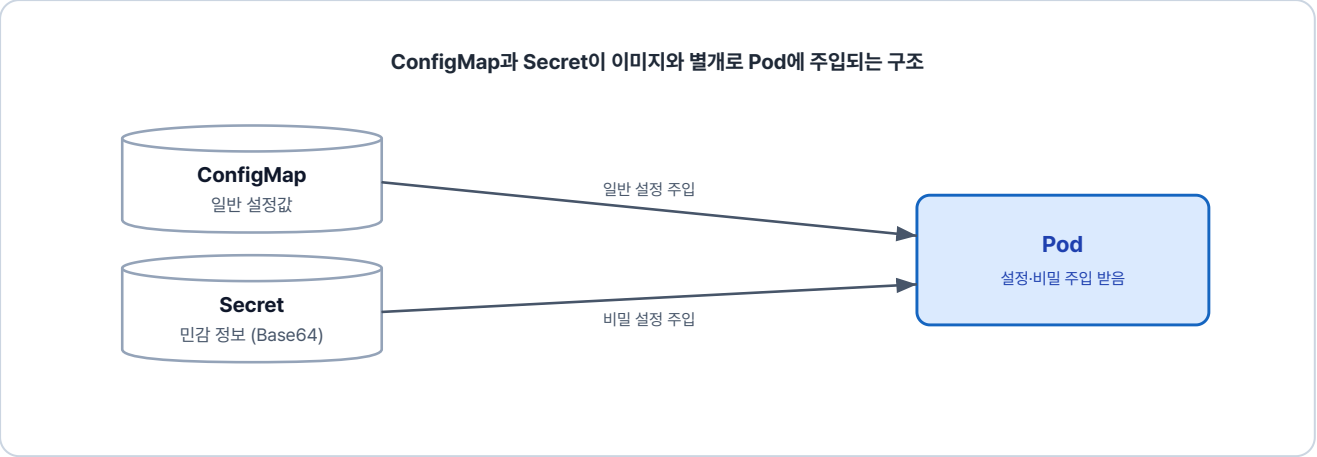


그림 6-3. ConfigMap과 Secret이 이미지와 별개로 Pod에 설정과 민감 정보를 주입

6.1.1 ConfigMap

ConfigMap은 환경에 따라 달라지는 설정값을 Pod에 환경 변수로 주입하는 리소스입니다.

이번 예제의 실습 파일은 `ex13` 폴더에 있습니다.

```
ex13/
├── configmap-conn.yml    # 설정값을 담는 ConfigMap 정의
├── secret-password.yml   # 비밀번호를 담는 Secret 정의
├── deploy-ex03.yml       # ConfigMap·Secret을 주입받는 Deployment
└── README.md             # 실습 안내
```

다음은 설정값 두 개를 정의한 ConfigMap입니다.

실습 1 **ex13/configmap-conn.yml. ConfigMap 정의**

```
apiVersion: v1
kind: ConfigMap
```

```yaml
metadata:
  name: configmap-conn                # ConfigMap 이름 지정
data:                                 # 설정값 넣는 영역
  conn_info: "localhost:80"
  conn_url: "config.test"
```

이어서 Deployment에 이 ConfigMap을 연결합니다. `envFrom.configMapRef`를 쓰면 ConfigMap의 값을 환경 변수로 주입받을 수 있습니다.

실습 2 **ex13/deploy-ex03.yml. ConfigMap을 Pod에 주입**

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-config-secret           # Deployment 이름 (재시작 시 이 이름으로 지목)
spec:
  replicas: 1                         # pod 수 지정
  selector:
    matchLabels:
      app: nginx                      # 이 라벨을 가진 Pod를 관리 대상으로
  template:
    metadata:
      labels:
        app: nginx                    # pod에 붙일 라벨
    spec:
      containers:
        - name: nginx-container
          image: nginx:1.20
          envFrom:
            - configMapRef:
                name: configmap-conn  # ConfigMap 연결
```

두 YAML을 적용한 뒤 Pod 안의 환경 변수를 확인합니다.

터미널 **ConfigMap·Deployment 적용과 환경 변수 확인**

```
kubectl apply -f ex13/configmap-conn.yml
kubectl apply -f ex13/deploy-ex03.yml
kubectl get pod                                                  # Pod 이름 확인
kubectl exec -it nginx-config-secret-794499d5d4-c2xmw -- env  # Pod 환경 변수 조회
```

실행 결과

```
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
nginx-config-secret-794499d5d4-c2xmw 1/1 Running 0 11s
$ kubectl exec -it nginx-config-secret-794499d5d4-c2xmw -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/bin
HOSTNAME=nginx-config-secret-794499d5d4-c2xmw
conn_info=localhost:80
conn_url=config.test
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT=tcp://10.96.0.1:443
```

그림 6-4. Pod 안의 환경 변수 목록에 ConfigMap의 값이 보이는 모습

ConfigMap에 적어 둔 conn_info와 conn_url이 Pod의 환경 변수로 주입됐습니다.

6.1.2 Secret

앞에서 본 ConfigMap이 일반 설정값을 위한 리소스라면, **Secret**은 비밀번호·토큰·키처럼 민감한 정보를 위한 리소스입니다.

다음은 비밀번호를 정의한 Secret YAML입니다. `stringData`에 평문을 적으면 쿠버네티스가 저장할 때 Base64로 인코딩합니다.

실습 3 **ex13/secret-password.yml. Secret 정의**

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: secret-password
stringData:
  password: metacoding1234
```

Secret을 클러스터에 적용한 뒤 저장된 값을 확인합니다. 아래 명령어를 실행하면 저장된 Secret의 전체 내용이 YAML 형태로 출력됩니다.

터미널 **Secret 적용 후 저장값 확인**

```
kubectl apply -f ex13/secret-password.yml      # Secret YAML 적용
kubectl get secret secret-password -o yaml     # 저장된 Secret 내용을 YAML로 조회
```

실행결과

```
$ kubectl get secret secret-password -o yaml
apiVersion: v1
data:
  password: bWV0YWNvZGluZzEyMzQ=
kind: Secret
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Secret","metadata":{"annotations":{},"name":"secret-
password","namespace":"defaul...
  creationTimestamp: "2026-03-15T06:30:27Z"
  name: secret-password
  namespace: default
  resourceVersion: "1387"
```

그림 6-5. Secret 내부를 보면 비밀번호가 Base64로 인코딩된 상태

저장된 결과를 보니 `password` 값이 Base64로 인코딩되어 알아볼 수 없는 문자열로 바뀌어 있습니다.

Pod에 주입하는 방식은 ConfigMap과 동일합니다. `deploy-ex03.yml`의 `envFrom` 아래에 `secretRef`를 추가하면, 쿠버네티스가 실행 시점에 Base64를 디코딩해 환경 변수로 주입합니다.

실습 4 **ex13/deploy-ex03.yml. Secret 연결 (envFrom 부분)**

```
envFrom:
  - configMapRef:
      name: configmap-conn
  - secretRef:
      name: secret-password   # Secret 연결
```

변경한 Deployment를 다시 적용한 뒤 환경 변수를 확인합니다.

터미널 **변경한 Deployment 적용과 환경 변수 확인**

```
kubectl apply -f ex13/deploy-ex03.yml                          # 변경한 Deployment 적용
kubectl get pod                                                # Pod 이름 확인
kubectl exec -it nginx-config-secret-7fbccb65f5-zq8nz -- env   # Pod 환경 변수 조회
```

실행결과

```
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
nginx-config-secret-7fbccb65f5-zq8nz 1/1 Running 0 51s
$ kubectl exec -it nginx-config-secret-7fbccb65f5-zq8nz -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/bin
HOSTNAME=nginx-config-secret-7fbccb65f5-zq8nz
TERM=xterm
conn_url=config.test
password=metacoding1234
conn_info=localhost:80
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
```

그림 6-6. 환경 변수 목록에서 확인한 Secret의 값

6.1.3 ConfigMap 변경

기본 동작을 확인했다면 다음은 ConfigMap을 변경해 보겠습니다. `configmap-conn.yml`의 `conn_info` 포트를 80에서 90으로 변경합니다.

실습 5 **ex13/configmap-conn.yml. 포트 변경**

```
# ... 생략
data:                                  # 설정값 넣는 영역
  conn_info: "localhost:90"            # 환경변수 변경
  conn_url: "config.test"
```

변경한 ConfigMap을 클러스터에 적용합니다.

터미널 **변경된 ConfigMap 적용 후 환경 변수 확인**

```
kubectl apply -f ex13/configmap-conn.yml                       # 변경된 ConfigMap 적용
```

```
kubectl get pod                                                  # Pod 이름 확인
kubectl exec -it nginx-config-secret-7fbccb65f5-zq8nz -- env  # Pod 환경 변수 조회
```

실행 결과

```
$ kubectl apply -f ex13/configmap-conn.yml
configmap/configmap-conn configured
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
nginx-config-secret-7fbccb65f5-zq8nz 1/1 Running 0 6m
$ kubectl exec -it nginx-config-secret-7fbccb65f5-zq8nz -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/bin
HOSTNAME=nginx-config-secret-7fbccb65f5-zq8nz
conn_info=localhost:80
conn_url=config.test
password=metacoding1234
KUBERNETES_SERVICE_HOST=10.96.0.1
```

그림 6-7. ConfigMap 적용 직후 기존 Pod의 환경 변수 조회 결과

ConfigMap은 갱신됐지만, 이미 실행 중인 Pod의 환경 변수는 여전히 80입니다.

'어? 분명히 바꿨는데 왜 그대로지?'

리눅스에서 환경 변수는 **프로세스가 시작될 때 한 번 주입되는 값**입니다. 그래서 ConfigMap이 갱신되어도 이미 실행 중인 Pod의 프로세스에는 처음 주입된 값이 그대로 남아 있습니다. **새 값을 반영하려면 Pod를 다시 실행해야 합니다.**

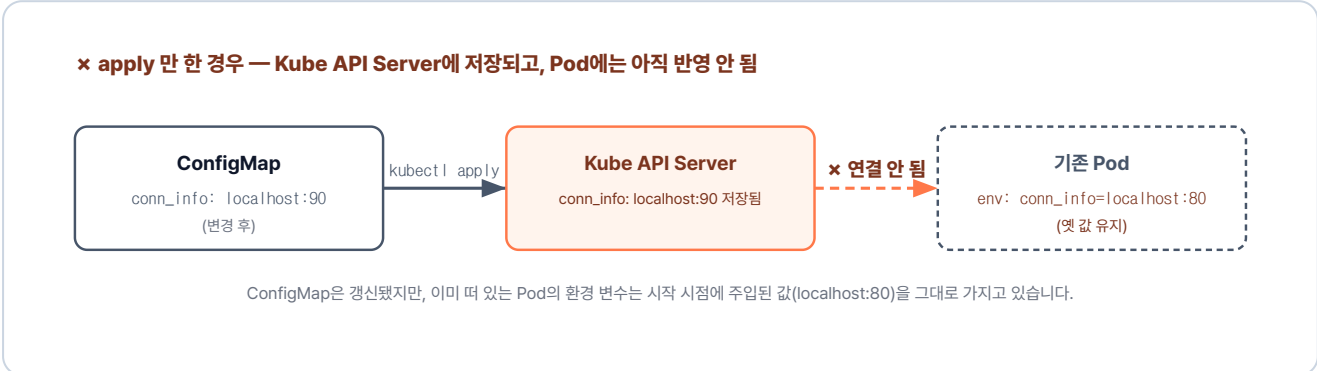


그림 6-8. apply 만 한 경우 - ConfigMap 변경이 기존 Pod에는 반영되지 않음

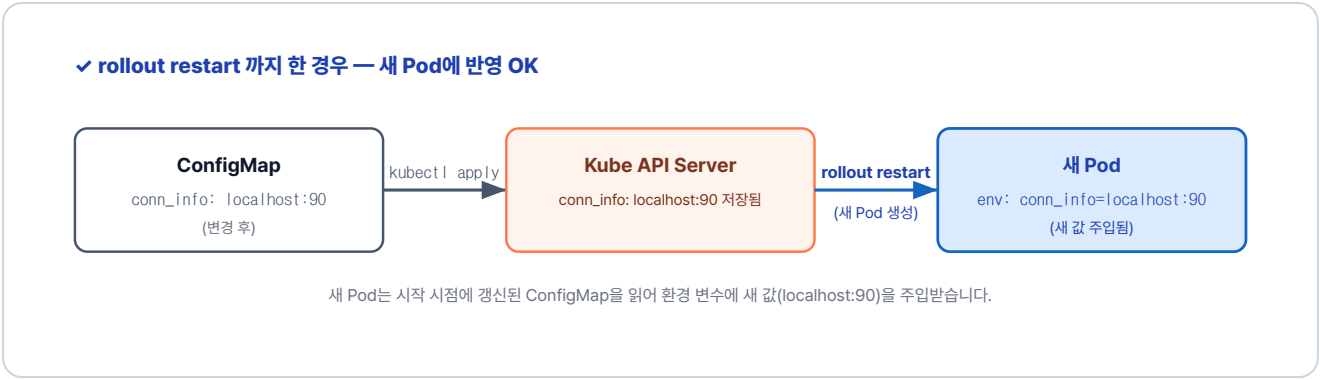


그림 6-9. rollout restart 로 Pod를 새로 실행하면 ConfigMap 새 값이 환경 변수로 반영됨

`kubectl rollout restart`는 Pod를 새로 교체해 새 값을 반영하는 명령입니다. 명령어를 실행해 Deployment를 재시작합니다.

터미널 **Pod 재시작 후 환경 변수 확인**

```
kubectl rollout restart deployment nginx-config-secret       # Pod 재시작
kubectl get pod                                              # 새 Pod 이름 확인
kubectl exec -it nginx-config-secret-8c5d4f9b6-r2t4p -- env  # Pod 환경 변수 조회
```

실행결과

```
$ kubectl rollout restart deployment nginx-config-secret
deployment.apps/nginx-config-secret restarted
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
nginx-config-secret-8c5d4f9b6-r2t4p 1/1 Running 0 12s
$ kubectl exec -it nginx-config-secret-8c5d4f9b6-r2t4p -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/bin
HOSTNAME=nginx-config-secret-8c5d4f9b6-r2t4p
conn_url=config.test
conn_info=localhost:90
password=metacoding1234
```

그림 6-10. Pod 재시작 후 환경 변수에 새 포트 90이 반영된 모습

재시작 후 환경 변수가 적용된 것을 확인할 수 있습니다.

설정은 Pod가 새로 만들어질 때 반영된다

쿠버네티스에서 Pod는 한 번 생성되면 그 설정 내용을 중간에 수정할 수 없는 불변성을 가집니다. 따라서 **kubectl apply**로 ConfigMap이나 Secret의 값만 변경하더라도, 이미 실행 중인 기존 Pod에는 새 값이 자동으로 반영되지 않습니다.

반면, Deployment 내의 Pod 템플릿(template)을 수정하여 적용하면 쿠버네티스는 이를 새로운 배포로 인식합니다. 즉시 교체 작업이 시작되어 기존 Pod를 종료하고 바뀐 템플릿으로 새로운 Pod를 생성하므로 변경 내용이 반영됩니다.

6.2 Volume - 데이터의 영속성 확보

6.2.1 Pod의 휘발성 문제

Deployment의 자동 복구가 데이터베이스에서는 다른 문제를 만듭니다. Pod는 기본적으로 **휘발성**이라, 안에서 만든 파일은 Pod 수명과 함께 사라집니다. 데이터가 매일 쌓이는 DB가 이대로 휘발되면 운영할 수 없습니다.

이 문제는 데이터를 Pod 외부에 두어 해결할 수 있습니다. Docker에서 본 마운트처럼, 쿠버네티스에서도 별도의 저장 공간에 데이터를 두는 기능을 **Volume**이라고 합니다.

Volume에는 여러 종류가 있습니다.

종류	설명	데이터 유지
`emptyDir`	Pod 생성 시 만들어지는 임시 저장 공간	Pod 삭제 시 함께 삭제
`hostPath`	워커 노드(호스트)의 특정 경로를 Pod에 마운트	노드에 남지만, Pod가 다른 노드로 이동하면 접근 불가
`PV / PVC`	클러스터가 관리하는 영구 저장소를 만들고 요청서(PVC)로 Pod에 연결	Pod가 삭제되어도 유지

이 중 가장 보편적으로 사용하는 **PV(PersistentVolume)** 와 **PVC(PersistentVolumeClaim)** 를 알아보겠습니다.

6.2.2 PV와 PVC

쿠버네티스는 저장소(PV)와 사용 요청(PVC)을 분리합니다. **PV는 용량과 권한 등 실제 저장 공간의 사양을 정의**하고, **PVC는 필요한 공간을 요구하는 신청서**입니다. PVC로 "1Gi 크기의 읽기·쓰기가 가능한 공간"을 요청하면, 쿠버네티스가 조건에 맞는 PV를 찾아 자동으로 연결합니다.

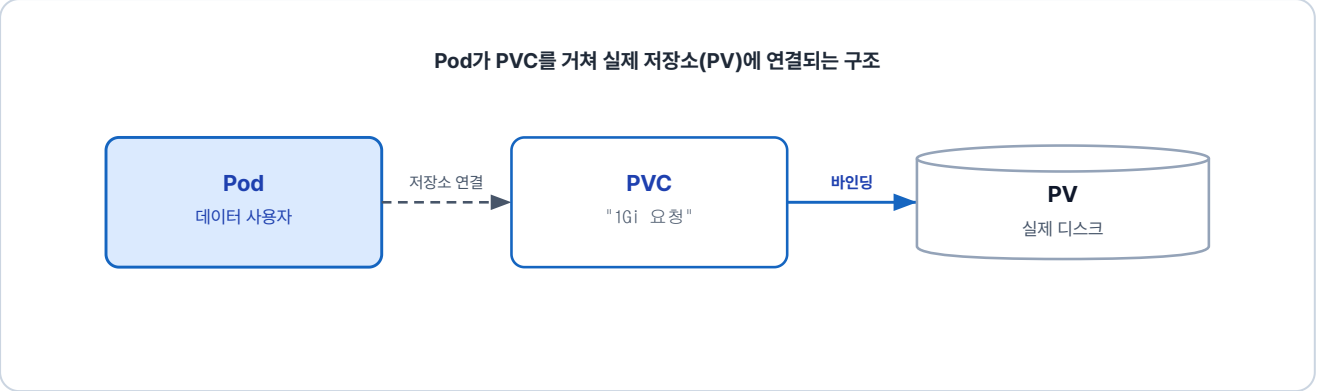


그림 6-11. PV는 실제 저장 공간, PVC는 그 공간을 요청하는 신청서

Pod는 PV를 직접 참조하지 않고 PVC만 연결해 사용합니다. 덕분에 실제 저장소가 무엇인지 몰라도 됩니다.

6.2.3 PV 만들기

PV를 먼저 만들어 보겠습니다. 이번 실습에서는 PV의 저장 위치로 Minikube 내부의 경로(`/mnt/data`)를 지정합니다.

이번 예제의 실습 파일은 `ex14` 폴더에 있습니다.

```
ex14/
├── volume-pv.yml      # 저장 공간을 제공하는 PersistentVolume 정의
├── volume-pvc.yml     # 저장 공간을 요청하는 PersistentVolumeClaim 정의
├── volume-pod.yml     # PVC를 마운트해 데이터를 쓰는 Pod 정의
└── README.md          # 실습 안내
```

실습 6 **ex14/volume-pv.yml. PersistentVolume 정의**

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: volume-pv                    # PVC가 volumeName으로 참조할 이름
spec:
  capacity:
```

```
    storage: 1Gi                        # 저장 용량
  accessModes:
    - ReadWriteOnce                     # 단일 노드 읽기·쓰기 모드
  storageClassName: ""                  # 자동 StorageClass 비활성 (정적 바인딩)
  hostPath:
    path: /mnt/data                     # Minikube 내부의 실제 저장 경로
    type: DirectoryOrCreate             # 경로가 없으면 자동 생성
```

> **StorageClass란**
> **StorageClass**는 PVC가 요청한 조건에 맞는 PV를 자동으로 만들어 주는 템플릿입니다. AWS·GCP·Minikube 같은 환경마다 기본 StorageClass가 따로 준비되어 있어 평소에는 PVC만 작성해도 PV가 자동 발급됩니다. 하지만 이번 실습은 PV를 직접 만들어 보기 위해 **storageClassName: ""** 로 자동 생성을 비활성화했습니다.

6.2.4 PVC 만들기

다음으로 앞서 만든 PV에 연결할 PVC를 만들어 보겠습니다.

실습 7 **ex14/volume-pvc.yml. PersistentVolumeClaim 정의**

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: volume-pvc                      # PVC 이름 (Pod가 claimName으로 참조)
spec:
  accessModes:
    - ReadWriteOnce                     # PV와 일치해야 바인딩
  storageClassName: ""                  # PV와 일치해야 바인딩
  resources:
    requests:
      storage: 1Gi                      # 신청 용량 (PV capacity 이하)
  volumeName: volume-pv                 # 바인딩할 PV 이름을 수동 지정
```

PVC를 생성하면 PV와 연결됩니다. 다만 둘이 연결되려면 서로 맞아야 할 조건이 있습니다.

> **PVC와 PV가 연결되는 조건**
> PVC와 PV가 연결되려면 읽기·쓰기 방식(`accessModes`)과 스토리지 종류(`storageClassName`)가 일치해야 하며, PV의 용량이 PVC가 요청한 크기 이상이어야 합니다. 이 조건 중 하나라도 어긋나면 PVC는 짝을 찾지 못하고 **Pending** 상태에 머무릅니다.
> 단, `volumeName`으로 특정 PV를 직접 지정했다면 다른 조건이 모두 맞더라도 이름이 일치하는 PV와만 연결됩니다.

6.2.5 Pod에 마운트

이번 실습은 편의를 위해 Deployment 대신 Pod를 직접 생성합니다. 앞서 만든 PVC를 Pod에 연결하면, 컨테이너가 저장소를 쓸 수 있습니다.

실습 8 **ex14/volume-pod.yml. PVC를 마운트한 Pod**

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: volume-pod                    # Pod 이름
spec:
  containers:
  - name: nginx-volume                # 컨테이너 이름
    image: nginx                      # 사용할 이미지
    volumeMounts:
    - name: storage                   # 아래 volumes에서 정의한 이름과 일치
      mountPath: /data                # 컨테이너가 데이터를 읽고 쓰는 경로
  volumes:
  - name: storage                     # volumeMounts에서 참조할 이름
    persistentVolumeClaim:
      claimName: volume-pvc           # 연결할 PVC 이름
```

컨테이너가 보는 마운트 경로는 `/data`이고, Minikube 내부의 실제 저장 경로는 `/mnt/data`입니다. 컨테이너가 `/data`에 파일을 생성하면 PVC와 PV를 거쳐 최종적으로 Minikube의 `/mnt/data`에 저장됩니다.

아래 명령어를 실행해 클러스터에 적용합니다.

터미널 **PV·PVC·Pod 일괄 생성과 Bound 확인**

```bash
kubectl apply -f ex14/volume-pv.yml      # PV(저장소) 생성
kubectl apply -f ex14/volume-pvc.yml     # PVC(저장소 요청) 생성
kubectl apply -f ex14/volume-pod.yml     # PVC를 마운트한 Pod 생성
kubectl get pv,pvc -o wide               # PV·PVC가 Bound 됐는지 확인
```

실행 결과

```
$ kubectl get pv,pvc -o wide
NAME CAPACITY ACCESS MODES RECLAIM POLICY STATUS CLAIM S...
persistentvolume/volume-pv 1Gi RWO Retain Bound default/volume-pvc <unset>
NAME STATUS VOLUME CAPACITY ACCESS MODES STORAGECLASS VOLUMEATTRIBUTESCl...
persistentvolumeclaim/volume-pvc Bound volume-pv 1Gi RWO <unset> 9s
```

그림 6-12. PV와 PVC가 Bound 상태로 연결된 결과

STATUS가 Bound 상태라면 PV와 PVC가 정상적으로 연결됐다는 뜻입니다.

확인을 위해 Pod 내부에서 파일을 하나 만든 뒤, Pod를 삭제하고 다시 실행해 보겠습니다.

터미널 **파일 생성 후 Pod 재생성으로 데이터 보존 확인**

```
kubectl exec -it volume-pod -- /bin/bash      # Pod 안 bash 접속
touch /data/c.txt                              # 마운트 경로에 빈 파일 생성
exit                                           # Pod 셸 종료

kubectl delete pod volume-pod                  # 기존 Pod 삭제
kubectl apply -f ex14/volume-pod.yml           # 같은 PVC를 쓰는 Pod 재생성
kubectl exec -it volume-pod -- ls /data        # 새 Pod에서 c.txt가 남아 있는지 확인
```

실행 결과

```
$ kubectl exec -it volume-pod -- /bin/bash
root@volume-pod:/# touch /data/c.txt
root@volume-pod:/# exit
exit
$ kubectl delete pod volume-pod
pod "volume-pod" deleted
$ kubectl apply -f ex14/volume-pod.yml
pod/volume-pod created
$ kubectl exec -it volume-pod -- ls /data
c.txt
```

그림 6-13. Pod가 새로 생성됐는데도 c.txt가 그대로 남아 있는 모습

새 Pod에서도 `c.txt`가 그대로 보입니다. Pod는 삭제·재생성되어도 PV 안 파일은 그대로 남습니다.

6.3 통합 실습 - 쿠버네티스 위에 웹사이트 올리기

마지막으로 지금까지 익힌 리소스를 모아 하나의 프로젝트로 연결해 보겠습니다. 앞서 Docker Compose로 실행했던 프로젝트를 쿠버네티스 위에 다시 구축합니다.

6.3.1 전체 그림

이번 실습의 구성도에는 프론트엔드, 백엔드, DB, 그리고 방문 횟수를 기록하는 Redis까지 서비스 네 개가 들어 있습니다. 앞서 Docker Compose로 실행했던 세 서비스에 Redis가 더해진 구조입니다.

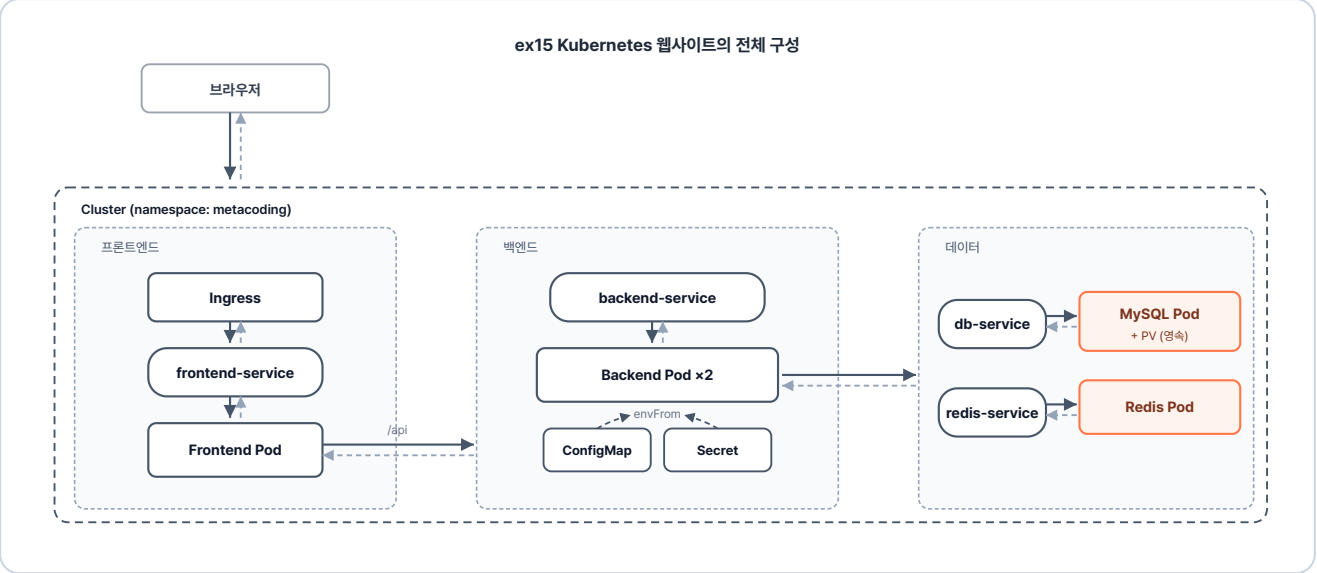


그림 6-14. ex15 Kubernetes 웹사이트의 전체 구성

6.3.2 폴더 구조와 진행 방식

ex15는 이미지를 만드는 폴더들과 쿠버네티스 설정(YAML) 폴더로 나뉘어 있습니다.

```
ex15/
├── backend/                    # Spring Boot 백엔드 이미지
│   ├── Dockerfile              # JDK 이미지 + entrypoint.sh 복사
│   └── entrypoint.sh           # Git clone + Gradle 빌드 + JAR 실행
├── db/                         # MySQL 이미지
│   ├── Dockerfile              # MySQL 이미지 + init.sql 복사
│   └── init.sql                # 테이블·초기 데이터 생성 스크립트
├── frontend/                   # NGINX + HTML 이미지
│   ├── Dockerfile              # nginx 이미지 + index.html·nginx.conf 복사
│   ├── index.html              # 로그인/게시판 UI (방문 카운터 표시)
│   └── nginx.conf              # /api 경로를 backend-service로 프록시
├── redis/                      # Redis 이미지
```

```
│   └── Dockerfile                    # redis 공식 이미지 기반
├── k8s/                              # 쿠버네티스 리소스 YAML
│   ├── namespace.yml                 # ex15 네임스페이스 정의
│   ├── backend/
│   │   ├── backend-configmap.yml     # 비밀이 아닌 설정값
│   │   ├── backend-deploy.yml        # 백엔드 Deployment
│   │   ├── backend-secret.yml        # DB 비밀번호 등 민감 정보
│   │   └── backend-service.yml       # 내부용 ClusterIP Service
│   ├── db/
│   │   ├── db-deploy.yml             # MySQL Deployment
│   │   ├── db-pv.yml                 # PersistentVolume (노드 로컬 저장소)
│   │   ├── db-pvc.yml                # PersistentVolumeClaim (볼륨 요청)
│   │   ├── db-secret.yml             # MySQL 계정 정보
│   │   └── db-service.yml            # 내부용 ClusterIP Service
│   ├── frontend/
│   │   ├── frontend-deploy.yml       # 프론트 Deployment
│   │   ├── frontend-ingress.yml      # 외부 진입점 (Ingress)
│   │   └── frontend-service.yml      # 내부용 ClusterIP Service
│   └── redis/
│       ├── redis-deploy.yml          # Redis Deployment
│       └── redis-service.yml         # 내부용 ClusterIP Service
└── README.md                         # 실습 안내
```

6.3.3 리소스 살펴보기

Namespace - 논리적 분리

지금까지 만든 리소스는 별도 공간을 지정하지 않았기 때문에 모두 **default라는 기본 공간**에 생성되었습니다. 혼자 실습할 때는 문제가 없지만, 여러 팀이 같은 클러스터를 함께 사용하면 리소스 이름이 겹쳐서 충돌할 수 있습니다.

하나의 회사 안에서 팀마다 사무실을 나눠 쓰듯이, 쿠버네티스도 리소스 공간을 분리할 수 있습니다. 이렇게 하나의 클러스터를 논리적으로 나눠주는 가상 공간을 **네임스페이스**(Namespace)라고 합니다.

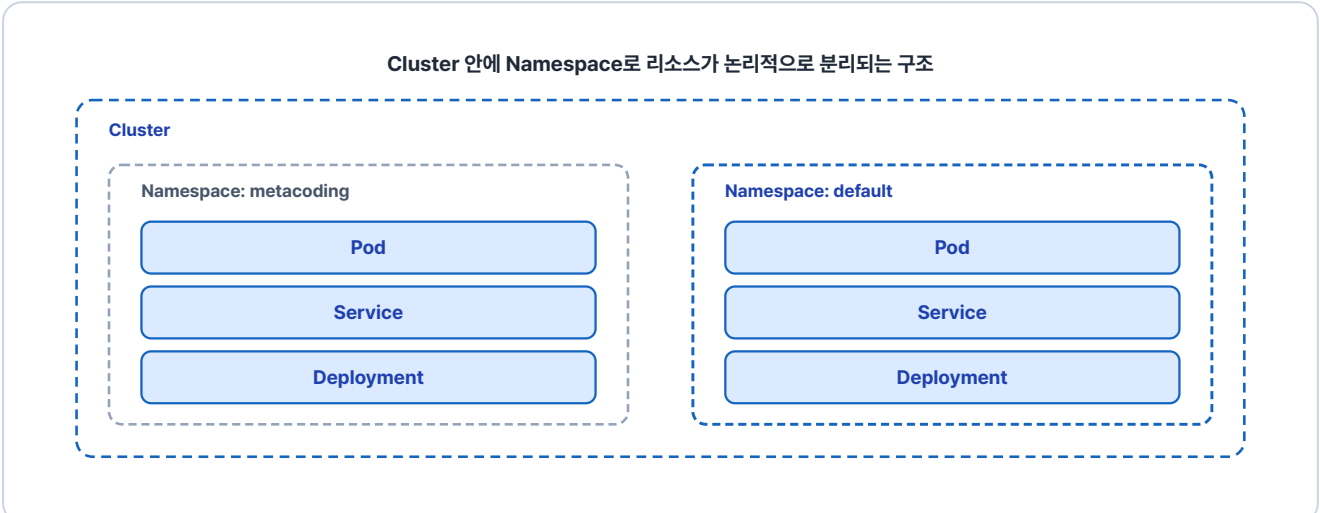


그림 6-15. 같은 클러스터 안에서 Namespace가 리소스를 논리적으로 분리

이번 실습에서는 `metacoding`이라는 전용 네임스페이스를 만들어 모든 리소스를 그 안에서 관리합니다.

실습 9 **ex15/k8s/namespace.yml. 네임스페이스 정의**

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: metacoding
```

이후 만들어지는 모든 리소스는 metacoding이라는 독립된 영역에 들어갑니다.

Frontend - 외부 진입

frontend 폴더는 외부에서 들어오는 요청이 처음 도달하는 입구를 정의합니다. 그 입구에서 요청을 어디로 보낼지는 `frontend-ingress.yml`이 정합니다.

실습 10 **ex15/k8s/frontend/frontend-ingress.yml. 외부 진입 Ingress**

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: frontend-ingress
  namespace: metacoding
spec:
  ingressClassName: nginx       # 어느 Ingress 컨트롤러가 이 규칙을 집행할지 지정
  rules:
    - http:
        paths:
          - path: /                 # 모든 경로를 잡음
            pathType: Prefix
            backend:
              service:
                name: frontend-service
                port:
                  number: 80
```

frontend-ingress.yml은 `/` 경로로 들어오는 모든 요청을 frontend-service로 넘깁니다.

같은 폴더의 나머지 두 파일은 다음과 같습니다.

파일	역할
`frontend-deploy.yml`	웹 화면을 보여주는 프론트엔드를 실행합니다
`frontend-service.yml`	Ingress가 프론트엔드 Pod로 들어가는 진입점입니다

Backend - API 처리

backend 폴더의 핵심 YAML은 `backend-deploy.yml` 입니다. Pod를 2개 생성하고, DB 접속 정보는 ConfigMap·Secret에서 가져옵니다.

실습 11 **ex15/k8s/backend/backend-deploy.yml. ConfigMap·Secret 주입**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-deploy
  namespace: metacoding              # 모든 리소스를 metacoding 네임스페이스에 배치
spec:
  replicas: 2                        # Pod 두 개로 트래픽 분산
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend-server
          image: metacoding/backend:1
          ports:
            - containerPort: 8080    # Tomcat 기본 포트
          envFrom:
            - configMapRef:
                name: backend-configmap   # DB 주소·JDBC URL·Redis 호스트 등 일반 설정
            - secretRef:
                name: backend-secret      # DB 계정·비밀번호
```

같은 폴더의 다른 세 파일은 다음과 같습니다.

파일	역할
`backend-service.yml`	다른 Pod가 백엔드 Pod로 들어가는 진입점입니다

파일	역할
`backend-configmap.yml`	환경에 따라 달라지는 설정값(DB·Redis 주소 등)을 담습니다
`backend-secret.yml`	비밀번호 같은 민감 정보를 담습니다

DB - 영속 저장소

db 폴더의 핵심 YAML은 `db-deploy.yml` 입니다. Pod 1개를 생성하고, 데이터는 PV에 저장합니다.

실습 12 **ex15/k8s/db/db-deploy.yml. PVC 마운트 MySQL Deployment**

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: db-deploy
  namespace: metacoding
spec:
  replicas: 1                        # DB는 단일 인스턴스로 운영
  selector:
    matchLabels:
      app: db
  template:
    metadata:
      labels:
        app: db
    spec:
      containers:
        - name: db-server
          image: metacoding/db:1
          ports:
            - containerPort: 3306    # MySQL 기본 포트
          envFrom:
            - secretRef:
                name: db-secret      # MySQL 계정·비밀번호
          volumeMounts:
            - name: data
              mountPath: /var/lib/mysql   # MySQL이 데이터를 쓰는 경로를 PV로
      volumes:
        - name: data
          persistentVolumeClaim:
            claimName: db-pvc
```

같은 폴더의 나머지 네 파일은 다음과 같습니다.

파일	역할
`db-service.yml`	백엔드가 DB Pod로 들어가는 진입점입니다
`db-secret.yml`	DB 접속 계정 정보를 담습니다
`db-pv.yml`	노드의 실제 저장 공간입니다
`db-pvc.yml`	DB가 저장 공간을 신청하는 요청서입니다

Redis - 인메모리 캐시

redis 폴더는 백엔드가 방문 횟수를 기록하기 위해 호출하는 인메모리 저장소이므로 다른 설정 파일 없이 두 파일로 구성됩니다.

파일	역할
`redis-deploy.yml`	방문 횟수를 기록하는 인메모리 저장소를 실행합니다
`redis-service.yml`	백엔드가 Redis Pod로 들어가는 진입점입니다

6.3.4 공통 준비

코드를 살펴봤으니 배포할 차례입니다. 실습을 시작하기 전에 Minikube와 Ingress 컨트롤러가 켜져 있지 않다면 아래 명령어를 실행합니다. 그런 다음 서비스별 이미지를 빌드합니다.

터미널 **Minikube와 Ingress 컨트롤러 시작**

```
minikube start
minikube addons enable ingress
```

실행결과

```
$ minikube addons enable ingress
* ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at:
https://github.com/kubernetes/minikube/blob/master/OWNERS
* After the addon is enabled, please run "minikube tunnel" and your ingress resources would
be available at "127.0.0.1"
   - Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
   - Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
   - Using image registry.k8s.io/ingress-nginx/controller:v1.13.2
* Verifying ingress addon...
* The 'ingress' addon is enabled
```

그림 6-16. Nginx Ingress 컨트롤러 애드온 설치

Minikube는 독립적인 가상 환경에서 작동하는 클러스터라, 로컬 호스트에서 빌드한 이미지를 곧장 인식하지 못합니다. 그래서 별도의 레지스트리를 두지 않고 `minikube image build` 명령으로 Minikube 자체에 이미지를 직접 만들어 둡니다.

터미널 **네 이미지 한 번에 빌드**

```
minikube image build -t metacoding/db:1 ex15/db              # DB 이미지 빌드
minikube image build -t metacoding/backend:1 ex15/backend    # 백엔드 이미지 빌드
minikube image build -t metacoding/frontend:1 ex15/frontend  # 프론트엔드 이미지 빌드
minikube image build -t metacoding/redis:1 ex15/redis        # Redis 이미지 빌드
```

6.3.5 한 번에 배포하고 결과 확인

리소스 구경을 마쳤으니 이제 `ex15/k8s/` 폴더를 한꺼번에 배포합니다.

터미널 **전체 리소스 일괄 배포**

```
kubectl apply -f ex15/k8s/namespace.yml      # Namespace 먼저
kubectl apply -f ex15/k8s/ --recursive       # k8s 이하 폴더의 리소스 전부 일괄 배포
```

명령을 치면 모든 리소스가 한꺼번에 생성됩니다. 이어서 상태를 조회합니다.

터미널 **배포 결과 조회**

```
kubectl get deploy,pod,service,ingress -n metacoding   # 리소스 상태 일괄 조회
```

프론트엔드와 데이터베이스 Pod는 빠르게 Running 상태로 바뀝니다. 백엔드 Pod만 Gradle 빌드와 의존성 설치 때문에 시간이 더 걸립니다.

모든 Pod가 Running 상태가 되면, `minikube tunnel`을 실행합니다. 그러면 Ingress 입구가 `127.0.0.1`로 열립니다.

터미널 **외부 터널 개방**

```
minikube tunnel                          # 새 터미널에서 실행 (관리자 권한 요구)
```

브라우저에서 `http://127.0.0.1`로 접속해 결과를 확인합니다.



그림 6-17. Ingress를 거쳐 웹사이트가 화면에 응답

화면에는 데이터베이스에서 불러온 정보와 함께 방문 횟수가 표시됩니다. 새로고침을 할 때마다 숫자가 하나씩 올라가는데, 방문 횟수가 **Redis에 저장되어 공유**되기 때문에 **백엔드 Pod가 두 개여도 카운터가 함께 증가**합니다.

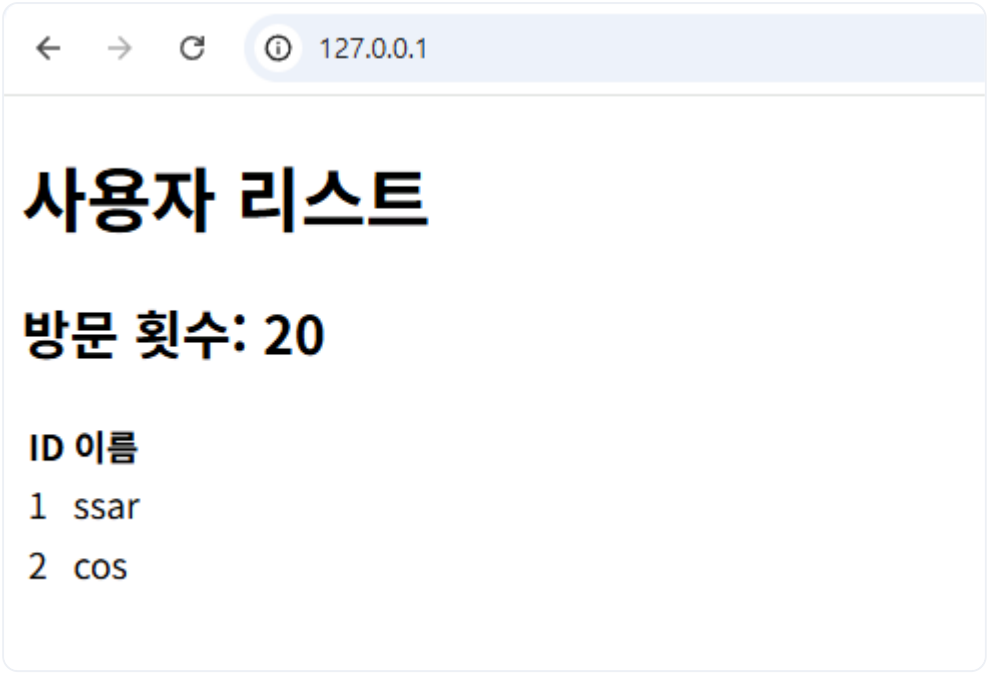


그림 6-18. 새로고침 시 방문 횟수가 증가

오픈이는 화면을 새로고침했습니다. 방문 횟수가 오르고, 사이트는 문제없이 응답합니다. 여러 서비스가 하나의 클러스터 안에서 각자 제 역할을 하며 돌아갑니다.

개발과 운영 환경의 차이를 극복하기 위해 시작된 여정은 자동화된 시스템 운영으로 완성됩니다. 코드를 환경째 컨테이너에 담아 어디서든 동일하게 실행할 수 있게 되면서, 인프라를 다루는 방식 자체가 바뀝니다. 장애가 난 컨테이너를 재시작하고, 부하에 맞춰 그 수를 늘리고, 중단 없이 배포하는 일까지 이제 온전히 클러스터의 몫이 됩니다.

이것만은 기억하자

- **설정과 비밀번호는 이미지 외부에 둡니다.** ConfigMap에는 환경마다 달라지는 일반 설정값을 두고, Secret에는 비밀번호·토큰 같은 민감 정보를 보관합니다.
- **ConfigMap을 바꾼 뒤에는 Pod를 새로 만듭니다.** 환경 변수는 프로세스 시작 시점에 한 번 꽂히는 값이라 `apply`만으로는 갱신이 반영되지 않습니다. `kubectl rollout restart`로 Pod를 교체해야 새 값이 박힙니다.
- **데이터는 PV·PVC로 Pod 외부에 두어 보존합니다.** Pod는 휘발성이라 안에서 만든 파일은 함께 사라지지만, PVC가 가리키는 PV에 데이터를 두면 Pod가 새로 생성돼도 그대로 남습니다.